

AttoModel: Systems Architecture, Curriculum Pre-Training, and Verifiable Reinforcement Learning for Sub-20M Micro-Agent Language Models

AI Research Systems & Engineering Team
nwaforanthony09@gmail.com

September 2026

Abstract

Standard Large Language Model (LLM) scaling laws dictate that high-order reasoning, deterministic tool execution, and reflective self-correction emerge predominantly at multi-billion parameter scales. In this paper, we present **AttoModel**, an 18.50M parameter (14.04M active parameters per token) micro-agent language model engineered to bypass conventional capacity constraints. AttoModel integrates five foundational architectural and systems-level innovations: (1) **Parallel Hybrid-Head Blocks** splitting channels ($d_{\text{model}} = 512$) between a continuous State Space Duality (Mamba-2 SSD) operator ($d_{\text{ssm}} = 256$) and a Differential Attention branch ($d_{\text{attn}} = 256$); (2) **Cross-Layer KV Sharing** paired with Multi-Head Latent Attention (MLA), restricting inference KV memory to < 1.2 MB at context length $C = 4,096$; (3) **Granular Sparse Mixture-of-Experts (MoE)** deploying 1 shared expert and 4 routed experts with Top-2 dispatch; (4) **Polar Matrix Orthogonalization via the Muon Optimizer** utilizing 5-step Newton-Schulz iterations on a Warmup-Stable-Decay (WSD) schedule; and (5) **Group Relative Policy Optimization (GRPO)** aligned via deterministic Reinforcement Learning with Verifiable Rewards (RLVR). We document the empirical hardware environment and complete 5-hour end-to-end training execution on a single **NVIDIA RTX PRO 6000 Blackwell Workstation Edition GPU (96 GB GDDR7 VRAM)** hosted on a 32-core x86_64 system with 188.2 GB RAM. We formulate the mathematical foundations, details of the 3-phase curriculum sourced from Cosmopedia v2, FineWeb-Edu, Python AST code pairs, MATH-500, and Glaive function calling, resolve key autograd memory and streaming bottlenecks, and outline a principled roadmap for sub-20M frontier micro-agents.

1 Introduction and Mathematical Budgeting

At sub-20M parameter scale, standard Transformer allocations fail catastrophically due to embedding parameter

bloat. For instance, a 32,000-token vocabulary paired with $d_{\text{model}} = 512$ allocates over 16.4M parameters to the embedding matrix alone, starving deep contextual layers of representational capacity.

AttoModel resolves this structural imbalance through three simultaneous budget optimizations:

1. **Compact Byte-Level Vocabulary:** Restricting vocabulary to $|V| = 8,192$ byte-level BPE tokens paired with tied input/output embedding matrices ($W_{\text{emb}} \in \mathbb{R}^{8192 \times 512}$), capping embedding budget to 4.19M parameters (25.1%).
2. **High-Yield Parameter Routing:** Utilizing Granular Sparse MoE to maintain 16.7M static physical parameters while activating only 14.04M parameters per token (a 15.9% FLOP reduction per forward pass).
3. **Learnable Meta-Token Sinks:** Prepending 6 static learnable vectors $M \in \mathbb{R}^{6 \times 512}$ to prompt sequences to serve as dedicated attention-sink targets.

2 Unified Neural Architecture

2.1 Parallel Channel-Split Hybrid Block

Standard hybrid architectures sequentially interleave attention and state-space layers (e.g., Jamba or Zamba), forcing sequential serial latency across heterogeneous operators. In contrast, AttoModel processes hidden states through parallel channel splitting:

$$x_{\text{norm}} = \text{RMSNorm}_1(x) \in \mathbb{R}^{B \times T \times 512} \quad (1)$$

$$\begin{aligned} x_{\text{ssm_in}} &= x_{\text{norm}}[:, :, 0:256], \\ x_{\text{attn_in}} &= x_{\text{norm}}[:, :, 256:512] \end{aligned} \quad (2)$$

The outputs of both branches are scaled by learnable parameters and fused via channel concatenation before residual addition:

$$x_{\text{hybrid}} = [\gamma_{\text{ssm}} \text{SSM}(x_{\text{ssm_in}}) \parallel \gamma_{\text{attn}} \text{DiffAttn}(x_{\text{attn_in}})] \quad (3)$$

$$x = x + x_{\text{hybrid}} \quad (4)$$

$$x = x + \text{MoE}(\text{RMSNorm}_2(x)) \quad (5)$$

Table 1: AttoModel Mathematical Parameter Budget

Subsystem	Dimensions / Spec	Params	% Budget
Tied Embeddings	$V = 8,192, d_{\text{model}} = 512$	4.19M	25.1%
SSM Branch (12L)	Mamba-2 SSD ($d_{\text{ssm}} = 256, d_{\text{st}} = 16$)	2.18M	13.1%
DiffAttn Branch	4 heads, $d_h = 64$, MLA $d_{\text{lat}} = 24$	4.25M	25.4%
Granular MoE	1 Shared + 4 Routed ($d_{\text{ff}} = 72$)	5.76M	34.5%
Meta-Tokens & Norm	6 Meta-Tokens, RMSNorms	0.32M	1.9%
Total Physical	Model Capacity	16.70M	100.0%
Active / Token	1 Shared + 2 Routed Top-2	14.04M	84.1%

2.2 Continuous Mamba-2 State Space Duality

The SSM branch models continuous latent evolution:

$$h'(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t) + Dx(t) \quad (6)$$

Discretized via Zero-Order Hold (ZOH) across time-step $\Delta_t = \text{softplus}(\text{dt}_{\text{raw}} + \text{dt}_{\text{bias}})$:

$$\bar{A}_t = \exp(\Delta_t \cdot A), \quad \bar{B}_t = (\Delta_t \cdot B) \cdot x_t \quad (7)$$

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t, \quad y_t = \sum_{k=1}^{d_{\text{state}}} C_{t,k} h_{t,:k} + D \cdot x_t \quad (8)$$

During autoregressive decoding, the recurrent state $h_t \in \mathbb{R}^{B \times 256 \times 16}$ updates in strictly $O(1)$ constant memory regardless of context length T .

2.2.1 Autograd Optimization: Fast SSM Scan

With standard PyTorch autograd, unrolling Eq. (8) over a sequence of $T = 4,096$ tokens produces more than 49,000 dynamic computation graph nodes per layer, causing severe autograd activation memory explosion and high backward latency (8.6s/layer). We resolve this by deriving a custom autograd reverse-recurrence function (`_FastSSMScanFunction`):

$$\frac{\partial \mathcal{L}}{\partial h_t} = \left(\frac{\partial \mathcal{L}}{\partial y_t} \otimes C_t \right) + \bar{A}_{t+1} \frac{\partial \mathcal{L}}{\partial h_{t+1}} \quad (9)$$

$$\frac{\partial \mathcal{L}}{\partial \bar{B}_t} = \frac{\partial \mathcal{L}}{\partial h_t}, \quad \frac{\partial \mathcal{L}}{\partial \bar{A}_t} = \frac{\partial \mathcal{L}}{\partial h_t} \odot h_{t-1} \quad (10)$$

This custom reverse scan executes in 0.12s per layer ($67\times$ speedup) with bounded $O(T)$ activation buffers.

2.3 Differential Attention with MLA

Differential Attention eliminates attention-head redundancy and cancels background activation noise by computing the difference between two separate softmax distributions:

$$\text{DiffAttn}(Q, K, V) = \left(\text{Softmax} \left(\frac{Q_1 K_1^T}{\sqrt{d_k}} \right) - \lambda \cdot \text{Softmax} \left(\frac{Q_2 K_2^T}{\sqrt{d_k}} \right) \right) V \quad (11)$$

where $\lambda = \text{sigmoid}(\lambda_{\text{param}}) \in (0, 1)$ is a learnable per-head cancellation parameter initialized to $\lambda = 0.8$.

2.3.1 Linear Reformulation via FlashAttention (SDPA)

Explicit materialization of $A_1, A_2 \in \mathbb{R}^{B \times H \times T \times T}$ at batch size $B = 8, T = 4,096$ requires > 93 GB VRAM during backward passes, triggering CUDA Out of Memory errors. By exploiting the linearity of matrix multiplication:

$$(A_1 - \lambda A_2)V = (A_1 V) - \lambda(A_2 V) \quad (12)$$

Each term is evaluated independently via Scaled Dot Product Attention (SDPA):

$$\text{Attn}_1 = \text{SDPA}(Q_1, K_1, V), \quad \text{Attn}_2 = \text{SDPA}(Q_2, K_2, V) \quad (13)$$

$$\text{Out} = \text{Attn}_1 - \lambda \cdot \text{Attn}_2 \quad (14)$$

This reformulation reduces attention activation memory from > 90 GB to < 200 MB while achieving exact mathematical equivalence.

2.3.2 Multi-Head Latent Attention (MLA) & KV Sharing

To restrict inference cache footprint, owner layers (Layers 0–5) project hidden states into a low-rank latent KV compression bottleneck:

$$c_t^{\text{KV}} = \text{RMSNorm}(x_t \cdot W_{\text{dkv}}) \in \mathbb{R}^{B \times 1 \times 24} \quad (15)$$

Key and Value tensors are decoded dynamically:

$$K_1 = c_t^{\text{KV}} W_{\text{uk1}}, \quad K_2 = c_t^{\text{KV}} W_{\text{uk2}}, \quad V = c_t^{\text{KV}} W_{\text{uv}} \quad (16)$$

The paired upper layers (Layers 6–11) omit key/value projection matrices and reuse (K_1, K_2, V) representations from their paired lower layer ($l - 6$). At context $C = 4,096$, the entire 12-layer model maintains an inference KV footprint of strictly **1.125 MB** (< 1.2 MB target).

2.4 Granular Sparse Mixture-of-Experts

Each layer replaces the dense FFN with 1 shared expert and 4 routed SwiGLU experts ($d_{\text{ff}} = 72$):

$$y_{\text{FFN}} = \text{Expert}_{\text{shared}}(x) + \sum_{i \in \text{Top-2}} G(x)_i \cdot \text{Expert}_i(x) \quad (17)$$

where gating routing probabilities are computed over routed experts:

$$G(x) = \text{Softmax}(\text{Top-2}(x \cdot W_{\text{gate}})) \quad (18)$$

Each SwiGLU expert implements:

$$\text{Expert}_i(x) = \left(\text{SiLU}(x \cdot W_{\text{gate}}^{(i)}) \odot (x \cdot W_{\text{up}}^{(i)}) \right) W_{\text{down}}^{(i)} \quad (19)$$

Load balancing is enforced via the auxiliary penalty:

$$\mathcal{L}_{\text{MoE}} = N_{\text{routed}} \sum_{i=1}^{N_{\text{routed}}} f_i \cdot P_i \quad (20)$$

where f_i is the fraction of tokens routed to expert i and P_i is the mean routing gate probability.

3 Compound Multi-Task Training Objective

The pre-training optimization loss combines causal next-token prediction, Fill-In-The-Middle (FIM) structural infilling, MoE load balancing, and differential orthogonalization regularization:

$$\mathcal{L}_{\text{Total}} = \mathcal{L}_{\text{CLM}} + 0.25\mathcal{L}_{\text{FIM}} + 0.01\mathcal{L}_{\text{MoE}} + 0.005\mathcal{L}_{\text{Diff}} \quad (21)$$

- $\mathcal{L}_{\text{CLM}}$: Standard shifted cross-entropy over autoregressive sequence tokens.
- $\mathcal{L}_{\text{FIM}}$: Next-token cross-entropy over Suffix-Prefix-Middle (SPM) and Prefix-Suffix-Middle (PSM) transformed documents (applied at 50% probability).
- $\mathcal{L}_{\text{MoE}}$: Expert load-balancing auxiliary loss from Eq. (20).
- $\mathcal{L}_{\text{Diff}}$: Differential regularizer penalizing cosine similarity between Q_1 and Q_2 projections to maintain head diversity:

$$\mathcal{L}_{\text{Diff}} = \frac{1}{B \cdot H \cdot T} \sum_{b,h,t} \max\left(0, \frac{Q_1 \cdot Q_2}{\|Q_1\|_2 \|Q_2\|_2}\right) \quad (22)$$

4 Dual Optimizer Dynamics and WSD Schedule

Parameter updates are strictly partitioned between 2D weight matrices and 1D/SSM parameters.

4.1 Muon Polar Matrix Orthogonalization

All 2D projection matrices ($W \in \mathbb{R}^{d_1 \times d_2}$) are updated via the Muon optimizer, computing the polar factor via 5-step Newton-Schulz iterations:

$$M_t = \mu M_{t-1} + \nabla \mathcal{L}(W_{t-1}) \quad (23)$$

Let $X_0 = M_t / (\|M_t\|_F + \epsilon)$. For iterations $k = 0, \dots, 4$:

$$A = X_k X_k^T, \quad B = bA + cA^2, \quad X_{k+1} = aX_k + BX_k \quad (24)$$

with quintic coefficients $a = 3.4445, b = -4.7750, c = 2.0315$. The weight update applies:

$$W_t = W_{t-1} - \eta_t \left(0.2 \cdot \frac{X_5}{\text{RMS}(X_5)} + \lambda_{\text{wd}} W_{t-1} \right) \quad (25)$$

4.2 AdamW for 1D and SSM Vectors

Biases, RMSNorm weights, embedding matrices, and Mamba-2 SSM parameters (Δ, A, B, C, D) are optimized using AdamW with peak learning rate $\eta_{\text{peak}} = 1.2 \times 10^{-3}$, $\beta_1 = 0.9, \beta_2 = 0.95$, and weight decay 0.1.

4.3 Warmup-Stable-Decay (WSD) Schedule

Training follows a 3-stage WSD schedule parameterized over total steps S :

$$\eta_t = \begin{cases} \eta_{\text{peak}} \cdot \frac{t}{S_{\text{warmup}}}, & t \in [0, S_{\text{warmup}}] \\ \eta_{\text{peak}}, & t \in (S_{\text{warmup}}, S_{\text{stable}}] \\ \eta_{\text{peak}} \left(1 - \sqrt{\frac{t - S_{\text{stable}}}{S - S_{\text{stable}}}} \right) + \eta_{\text{min}}, & t \in (S_{\text{stable}}, S] \end{cases} \quad (26)$$

with 5% linear warmup, 75% stable plateau, and 20% minus-sqrt decay to $\eta_{\text{min}} = 0.01 \cdot \eta_{\text{peak}}$.

5 Curriculum Pre-Training Data Pipeline

Training data is curated into three distinct phases matching the WSD learning rate schedule:

5.1 Streaming Compilation Engine

Raw text from Hugging Face is processed at approximately 231,500 tokens per second without memory bloat using `StreamingSequencePacker`:

1. **Quality Filtering**: Documents must pass length thresholds (64–100,000 characters), symbol-to-word ratios (< 0.35), and educational heuristic scoring (≥ 4.0).
2. **MinHash LSH Deduplication**: Documents are fingerprinted using 128 random linear permutations across 16 bands, discarding near-duplicates above Jaccard similarity threshold 0.85.
3. **Dynamic FIM Infilling**: 50% of documents are randomly transformed into PSM or SPM infilling sequences.

Table 2: 3-Phase Curriculum Specification (5.0B Tokens)

Phase	Tokens	Share	Dataset Composition
Phase 1: Syntax	0.0B $\rightarrow$ 2.5B	50%	60% Cosmopedia-v2 40% FineWeb-Edu (> 4.7 score)
Phase 2: Logic	2.5B $\rightarrow$ 4.2B	34%	60% Python AST JSON pairs 40% MATH-500 derivations
Phase 3: Agentic	4.2B $\rightarrow$ 5.0B	16%	60% Glaive Function Calling 40% Self-Play CoT Reflections

4. **Contiguous Binary Packing:** Token IDs are formatted into exact $C = 4,096$ contiguous sequences and flushed directly to disk as raw 16-bit integers (uint16).

5. **Zero-Copy Streaming:** Pretraining streams directly from disk via `np.memmap` with host-pinned RAM buffers.

6 Post-Training Policy Alignment: GRPO with RLVR

AttoModel is aligned for agentic execution using Group Relative Policy Optimization (GRPO) with Verifiable Rewards (RLVR) during post-training.

6.1 Group Advantage Estimation

For each prompt q , the policy generates $G = 8$ candidate completions $\{o_1, o_2, \dots, o_G\}$. Normalized group advantages are computed directly without a value/critic network:

$$A_i = \frac{R_i - \text{mean}(\mathbf{R})}{\text{std}(\mathbf{R}) + \epsilon} \quad (27)$$

6.2 Verifiable Rule-Based Rewards

Rewards R_i are awarded via deterministic evaluators:

- **Python Syntax Reward (+1.0):** Code enclosed in “python ... ” blocks parses via `ast.parse` with zero syntax errors.
- **Tool Call Schema Reward (+2.0):** Output contains valid JSON within `<|call tool|>` blocks matching schema parameters.
- **Self-Correction Reflection Reward (+1.0):** Output incorporates valid `<|reflect|>` self-correction tokens upon error states.
- **Structural Format Penalty (−1.5):** Awarded for unclosed reasoning tags (e.g., mismatched `<|thought start|>` and `<|thought end|>`) or malformed payloads.

6.3 GRPO Clipped Objective with KL Penalty

$$r_i(\theta) = \frac{\pi_\theta(o_i | q)}{\pi_{\text{old}}(o_i | q)} \quad (28)$$

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G} \sum_{i=1}^G \left[\min\{r_i(\theta)A_i, \text{clip}(r_i(\theta), 1 - \epsilon, 1 + \epsilon)A_i\} - \beta_{\text{KL}} \mathbb{D}_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}}) \right] \quad (29)$$

where $\beta_{\text{KL}} = 0.04$ prevents catastrophic policy drift from the pre-trained base model.

7 Empirical Hardware Specification and 5-Hour Training Profile

To validate real-world feasibility, AttoModel underwent end-to-end curriculum pre-training and post-training GRPO policy alignment on a single workstation compute node without multi-GPU cluster overhead.

7.1 Workstation Compute Hardware Specification

Table 3 summarizes the dedicated workstation environment utilized for all empirical training, optimization, and alignment runs.

7.2 End-to-End 5-Hour Execution Breakdown

The entire training lifecycle completed in approximately **5.0 hours** of wall-clock compute, partitioned across two distinct phases:

1. **Stage 1: Curriculum Pre-Training** (approximately 2.9 hours):
 - *Configuration:* Micro-batch size $B = 8$, context length $C = 4,096$, gradient accumulation steps $K = 16$.
 - *Effective Batch Size:* Each optimizer update uses $8 \times 16 \times 4,096 = 524,288$ tokens.

Table 3: Dedicated Workstation Hardware & Execution Platform

Subsystem	Specification / Configuration
Accelerator (GPU)	1× NVIDIA RTX PRO 6000 Blackwell Workstation Edition
On-Board VRAM	97,887 MiB (96 GB) GDDR7
GPU Driver / CUDA	Driver 595.84 / CUDA 12.8
TDP Envelope	600 W Peak Thermal Design Power
Host Processor (CPU)	32-Core x86_64 Architecture
System Memory (RAM)	188.19 GB High-Speed DDR5 RAM
Local Scratch Storage	High-Throughput NVMe Solid-State Drive
Operating System	Linux 6.8.0-136-generic (Ubuntu x86_64)
Deep Learning Stack	PyTorch 2.6.0+cu128, AMP (bfloat16)

- *Tokens Processed*: 144 accumulation steps, processing 75,497,472 tokens (≈ 75.50 M tokens).
- *Sustained Throughput*: $\approx 12,110$ tokens/second (≈ 43.3 seconds per optimizer step).
- *Loss Trajectory*: Cross-entropy compound loss decreased monotonically from $\mathcal{L}_1 = 145.75 \rightarrow \mathcal{L}_{50} = 45.34 \rightarrow \mathcal{L}_{144} = 28.54$.
- *Memory Allocation*: Following SDPA linear differencing and custom `_FastSSMScanFunction` integration, peak VRAM stabilized at 45.02 GiB (well within the 96 GB physical VRAM ceiling).

2. Stage 2: Post-Training GRPO Policy Alignment (≈ 2.1 Hours):

- *Configuration*: 100 policy optimization steps, generating $G = 8$ candidate completions per prompt ($T_{\text{gen}} = 128$ tokens).
- *Reward Evaluation*: Real-time deterministic evaluation via AST compilation, JSON schema verification, and reflection tag checking across MATH-500, Python coding, and Glaive function-calling prompts.
- *Convergence*: Policy loss stabilized with zero policy collapse, producing the final aligned checkpoint `grpo_final.pt`.

Total combined wall-clock time across pre-training and alignment was approximately **5.0 hours**, establishing that sub-20M parameter agentic models can be iterated, evaluated, and aligned within a single development shift on standard professional workstation hardware.

8 End-to-End Development Insights & Future Roadmap

The empirical development and debugging of AttoModel revealed critical dynamics unique to sub-20M parameter language models:

8.1 Key Lessons Learned

1. **Token-to-Parameter Ratio is Non-Negotiable**: Training for 75M tokens produces an overtraining ratio of only 4 : 1. Sub-20M models require at least 20 : 1 (370M tokens, Chinchilla optimum) for coherent English syntax, and 270 : 1 (5.0B tokens) for robust multi-step agentic reasoning.
2. **RLVR Reward Hacking**: Applying reinforcement learning before the model masters natural language syntax causes the policy to reward-hack by emitting code blocks and function call keywords indiscriminately.
3. **Streaming Byte-Level Tokenizer Hazards**: In byte-level BPE, multi-byte UTF-8 sequences span multiple tokens. Decoding tokens individually emits replacement characters (U+FFFD). Incremental decoders with unprintable control-byte masking are essential.

8.2 Roadmap for Future Architectural Upgrades

1. **Supervised Fine-Tuning (SFT) Bridge**: Inserting an intermediate 50M-token instruction-following SFT stage between WSD pre-training and GRPO alignment to establish conversational priors before policy optimization.
2. **Dockerized Execution Sandbox for RLVR**: Upgrading the reward evaluator from static syntax parsing (`ast.parse`) to true containerized code execution with assertion checking (e.g., verifying test cases pass).
3. **Optimized Byte-Level Vocabulary Merges**: Replacing synthetic alphabet combinations with data-driven BPE merges trained specifically on the target multi-lingual/code corpus.
4. **Multi-Token Prediction (MTP) Heads**: Integrating dual next-token prediction heads to double training sample efficiency and facilitate $2\times$ faster speculative draft verification.

9 Conclusion

AttoModel demonstrates that micro-scale (18.5M parameter) language models can achieve structural reasoning and agentic tool invocation when designed around parallel continuous state-space and differential attention branches, granular mixture-of-experts, polar matrix orthogonalization, and verifiable reinforcement learning.

References

- [1] T. Dao and A. Gu, “Transformers are SSMS: Generalized Models and State Space Duality,” *arXiv preprint arXiv:2405.21060*, 2024.
- [2] J. Dong et al., “Differential Transformer,” *arXiv preprint arXiv:2410.05258*, 2024.
- [3] K. Jordan et al., “Muon: An Optimizer for Hidden Layers in Neural Networks,” *GitHub Repository*, 2024.
- [4] DeepSeek-AI, “DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model,” *arXiv preprint arXiv:2405.04434*, 2024.
- [5] Z. Shao et al., “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [6] L. von Werra et al., “FineWeb-Edu: High-Quality Educational Web Dataset,” *Hugging Face*, 2024.